

Improving Automated Assessment

by
Michal Skraburski

In partial fulfilment of the
requirements for the degree of
Bachelor of Science
in
CS408: Individual Project



Department of
Computer and Information Sciences

August, 2026

Abstract

It should not exceed one page. Mention the scope and objectives of the project, the methodology used, the main findings, and significance of your results.

Acknowledgements

I would like to thank the participants who evaluated my project designs, my supervisor Andrew Fagan, the friends and family who supported me throughout.

Contents

1	Introduction	1
2	Background Literature	3
2.1	Automated Assessment of Programming	3
2.2	The Feedback Problem	3
2.3	Usability and Design Principles	4
2.4	Standards and Engineering Practice	4
2.5	Positioning of this Project	5
3	Specification & Design	6
3.1	Methodology	6
3.2	Analysis	6
3.2.1	Understanding the System	6
3.2.2	Environment Setup	8
3.2.3	Problem Identification	8
3.2.4	Review of the Author Page	8
3.2.5	Review of the Student Page	11
3.2.6	Survey Analysis	11
3.3	Requirements	13
3.3.1	Development Environment Requirements	13
3.3.2	System Functional Requirements	13
3.3.3	System Non-Functional Requirements	14
3.4	Design	14
3.4.1	Interface Design	14
3.4.2	System Design	17
4	Product	19
4.1	Implementation	19
4.1.1	Development Environment	19
4.1.2	Test Case Management	19
4.1.3	Precheck Rendering Fix	21
4.1.4	Save Draft Button	21
4.1.5	Layout Switcher	22
4.1.6	Info Panel Collapse	23
4.1.7	Accessibility	24
4.1.8	Documentation Page	25
4.1.9	Security and Personal Data	26
4.2	Verification & Validation	26
4.2.1	Verification	26
4.2.2	Validation	27
5	Results & Evaluation	29
5.1	Evaluation Process	29
5.2	Results of Evaluation	29
5.2.1	Reception of the Interface Changes	31
5.2.2	Documentation Findings	32
6	Discussion & Reflection	33

6.1	Interpreting the Results	33
6.2	Reflection	33
6.3	Challenges	34
6.4	Limitations	34
6.5	Future Work	35
7	Conclusion	36
A	Appendix	39
A.1	Ethical Approval Form	39
A.2	Participant Information Sheet	39
A.3	Consent Form	39

1 Introduction

Universities teach programming to cohorts far larger than any team of markers can assess by hand, and automated assessment has become the standard answer: students submit real code, the code is run against tests, and a grade is produced in seconds [4]. At the University of Strathclyde this happens inside Moodle, the online learning platform behind MyPlace, through a plugin called CodeRunner [8]. CodeRunner adds a new kind of quiz question: one answered by writing a program, which is compiled, executed and graded automatically. It is mature, widely deployed and, in terms of raw capability, largely complete.

Capability, however, is not the same as usability. The lecturers who author CodeRunner questions face a dense editing form offering a full suite of tools with little guidance on how to use them: test cases can be added only in fixed batches and never deleted, help text vanishes the moment it loses focus, controls sit misaligned, and the official manual is a single enormous page organised around the system's features rather than the author's task. The students answering questions have their own frictions: a fixed layout that forces scrolling between the question and the answer, screen space consumed by margins and panels, hidden tests that give zero details why they fail and no modern LSP support – using the CodeRunner page for writing and testing code execution is doom for the students' grade. None of these faults stops anyone using the tool. Each one, multiplied across every question authored and every answer written by thousands of users, quietly taxes teaching and learning alike. Well-established design principles hold that this kind of friction is a software problem rather than a user problem [14], and tools embedded in education must keep evolving to meet its growing demands [17].

This project set out to remove that friction. The problem was approached as a user experience engineering task on a live, mature codebase: understand the system and its two audiences, identify the faults with evidence rather than intuition, turn the evidence into requirements, and deliver the improvements without breaking anything that already worked. Concretely, the project analysed CodeRunner through heuristic review, a survey of students and teachers, and interviews; built a fully reproducible Docker-based development environment in which the entire system can be recreated with one command; and delivered a set of improvements spanning both pages: proper test case management for authors, a corrected Precheck display, a switchable and resizable question layout, reclaimable screen space, an on-demand save for students, measurable accessibility gains, and task-focused documentation served from inside the plugin itself. The changes were verified with unit and

browser-driven acceptance tests, and the delivered features were subsequently confirmed to be worth building by the survey, whose results are reported in Section 5.

The project's objectives were:

Objective 1: Construct an isolated, repeatable development environment in which Moodle, CodeRunner and its dependencies can be built, run and regression-tested with minimal effort.

Objective 2: Improve the ergonomics of the author page: flexible test case management, correct rendering, visual consistency and measurable accessibility.

Objective 3: Improve the student answering experience: control over the question layout, reclaimable screen space and the ability to save work on demand.

Objective 4: Provide effective, task-focused documentation reachable from within the tool.

Objective 5: Evaluate the delivered changes with real students and teachers, and derive further requirements from their feedback.

The remainder of this dissertation follows the project's own course. Section 2 reviews the literature that informed it. Section 3 describes the methodology, the analysis of CodeRunner and its users, the requirements that emerged, and the designs that answered them. Section 4 details the implementation of each change and the verification and validation performed. Section 5 presents the survey results and their evaluation. The report closes with a discussion of the findings, an honest reflection on the process, and the conclusions drawn from the work.

2 Background Literature

This section reviews the literature that informed the project, in four strands: the state of automated programming assessment, the field's persistent weakness in feedback, the design principles used to evaluate and improve the interfaces, and the standards and practices that shaped the engineering. Each strand ends where the project begins, and the review closes by positioning the project against them.

2.1 Automated Assessment of Programming

Assessing programming by hand does not scale, and it barely works even when it is affordable: Lobb and Harlow open the CodeRunner paper by observing that hand-written code on paper is nearly impossible to grade meaningfully, because static reading cannot reveal whether a student could make their program actually run [8]. The field's answer, developed over decades, is to execute submissions against instructor-defined test cases and grade the outcome. Ihantola et al.'s systematic review of assessment systems shows both the maturity and the fragmentation of this space: dozens of tools exist, most built around the same test-based core, differing in how teachers define tests, how resubmission is handled, and how securely untrusted code is run [4]. Two of that review's conclusions matter here. First, test-based assessment is the settled approach; CodeRunner is an orthodox member of the family. Second, the authors explicitly criticise the field's habit of building yet another new system, and encourage opening up and extending the systems that already exist. That recommendation is a direct argument for the shape of this project: improving a widely deployed tool rather than producing a new one.

Pieterse's analysis of automated assessment adds the operational perspective: the success of such systems depends on factors beyond grading correctness, including how students experience failure and how much confidence teachers have in the tool, and the requirements placed on these systems grow as education leans harder on them [17]. A tool can therefore be complete in capability yet still fall short of what its users need, which is precisely the position this project found CodeRunner in.

2.2 The Feedback Problem

The most consistent criticism in the assessment literature concerns feedback. Keuning, Jeuring and Heeren's systematic review of automated feedback generation finds that most tools report whether a submission is correct but offer little help with why it failed or how to move forward [5]. This

is the academic form of the complaint that surfaced independently in this project's survey: hidden test cases fail without telling the student what went wrong, leaving them to guess at the cost of further penalty. The literature thus both anticipates the pain point and elevates it from an anecdote to a known, field-wide weakness, which is why hints for hidden test cases (FR5) is treated as the highest-value item of future work rather than a passing suggestion.

A newer strand asks whether large language models can close the feedback gap. Phung et al. benchmark ChatGPT and GPT-4 against human tutors across programming education tasks, including grading, and find that even the strongest models approach but do not match human tutors [16]. For grading specifically, where consistency is a fairness requirement, approaching is not enough; this evidence underpins the decision, discussed in Section 3, to consider and exclude LLM-based grading from the project's scope.

2.3 Usability and Design Principles

The project's central claim, that CodeRunner's remaining problems are experience problems, rests on established design literature. Norman's *The Design of Everyday Things* argues that what is habitually called user error is usually design error: systems that punish slips, hide their state, or demand documentation to operate are failing their users, not the reverse [14]. Norman also supplies the vocabulary of affordances, controls whose appearance invites their use, which this project applies directly in the design of the Save Draft button. Krug's *Don't Make Me Think* distils the same stance for the web: an interface should be self-evident, and every moment a user spends puzzling over a control is a cost paid for a design decision [7]. These two works served as the lenses for the heuristic review of CodeRunner's pages reported in Section 3.

Written guidance is part of the interface as well. Google's developer documentation style guide asks writers to organise content around the reader's task, keep pages short and scannable, and explain no more than the task requires [2]. It was used as the yardstick for CodeRunner's documentation, and the requirement it produced (NFR3) is essentially the guide's advice turned into an acceptance criterion.

2.4 Standards and Engineering Practice

Accessibility work in this project is anchored to two W3C standards: the Web Content Accessibility Guidelines define what an accessible page must achieve [19], and WAI-ARIA defines the vocabulary of attributes used to achieve it in rich interfaces, including the labelling and live-region techniques

applied here [18]. The Lighthouse audit used to measure progress derives its accessibility checks from these guidelines, which makes the requirement NFR2 measurable rather than aspirational.

The engineering practices around the product likewise follow the literature. Docker's containerisation, packaging applications with their dependencies into isolated and reproducible units [10], is what makes the development environment requirements (DE1 and DE2) satisfiable with one command. Behaviour-driven development, as introduced by North, frames acceptance tests as executable descriptions of behaviour [15]; CodeRunner's upstream Behat suite is exactly such a description, and running it unmodified is this project's regression evidence. The survey instrument was designed with reference to Kitchenham and Pfleeger's principles of survey research in software engineering [6], and the validation of requirements against implementations follows the traceability practice analysed by Gotel and Finkelstein [3].

2.5 Positioning of this Project

Read together, the strands position the project precisely. The assessment literature says the grading problem is solved and advises improving existing systems over building new ones; the feedback literature identifies experience, not capability, as the open weakness; the design literature supplies principles and methods for fixing experience problems and insists such problems belong to the software; and the standards make the improvements measurable. This project sits at that intersection: an evidence-driven user experience improvement of a mature, widely used automated assessment tool, evaluated with its real users.

3 Specification & Design

This section describes how the project took shape: how CodeRunner and its place inside Moodle (the platform behind Strathclyde's MyPlace) came to be understood, how the state of the system was assessed, and how that assessment turned into a concrete set of requirements and designs. The work began by picking the system apart to learn how it operates: its components, its dependencies, and how they fit together. Its features were then inspected with a critical eye, hunting for flaws, bugs and missing functionality, with the opinions of real users gathered as a check on the analysis.

Doing any of this needed somewhere safe to experiment, so the first step was building a fresh, self-contained copy of the entire system: a Dockerized Moodle with CodeRunner and its dependencies integrated into it, plus scripts that automate compiling and testing. The four codebases involved – CodeRunner, its companion behaviour plugin, the Dockerized Moodle and this dissertation – live together in one shared repository (a monorepo), which kept the moving parts easy to manage as one project.

CodeRunner is a mature and largely feature-complete system, so the project's focus fell naturally on user experience: analysing, modifying and polishing the pages that lecturers and students actually touch. The features chosen for development came from the analysis and were confirmed to be worth building through a survey. The remainder of this section covers the methodology followed, the analysis itself, the requirements that emerged from it, and the designs that answered them.

3.1 Methodology

The project followed a technique similar to Agile (single 2-week sprint). Development ran as a closed feedback loop: upon a feature reaching a good state, opinions were requested from peers and the project supervisor, alongside consulting the literature reviewed in Section 2. This technique was chosen specifically because it fits an iterative style of design better than the other options. It also serves UI design well, as it benefits from a constant feedback loop.

3.2 Analysis

3.2.1 Understanding the System

It is first necessary to understand CodeRunner, its operation, and its integration with the wider Moodle platform. This was done by looking at the project, cloning the repo to the development environment and setting it up. Moodle itself is the core: an online education platform used by

universities to host courses, quizzes and teaching material, which allows modification by plugins, add-on packages that can alter scripts, override behaviour and styles, and add functionality. CodeRunner is a plugin which augments itself onto the list of available question types [8]. For example, a multiple choice question and a text entry question are both question types; CodeRunner expands this list of question types by one: a question answered by writing a real computer program, graded automatically by running it against test cases, example inputs paired with the outputs the lecturer expects, the approach taken by most automated assessment systems [4]. CodeRunner extends the question framework through a required dependency, `qbehaviour_adaptive_adapted_for_coderunner` (Figure 1). This required plugin is the railway for sending input, defining and managing the types of submission and their results (such as a Precheck, a trial run that lets a student check their code without spending marks); CodeRunner itself is the station for this railway, managing the interaction between Jobe (a separate, sealed-off server whose only job is to compile and run the code students submit, keeping untrusted code out of Moodle itself; deployed here as the ready-made Jobebox package) and the now-overridden types of results. CodeRunner extends the author page (where a lecturer builds a question) and the student question page (where a student answers it) through Moodle's standard extension points to add its functionality, as per the Moodle plugin dogma.

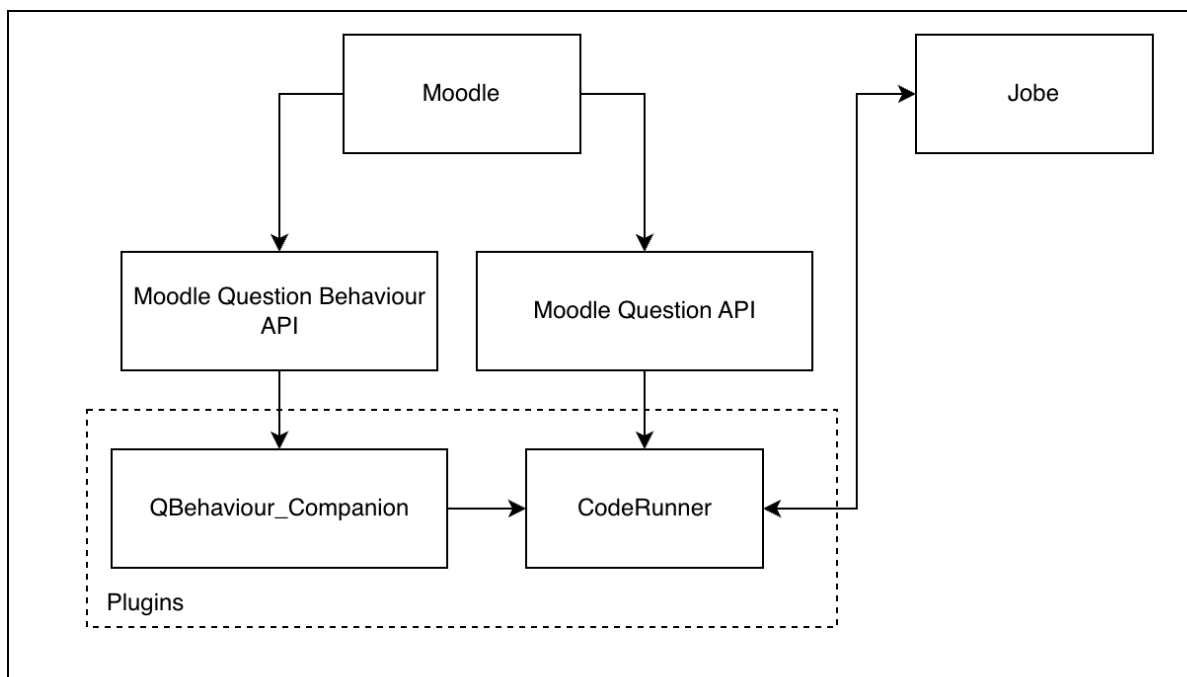


Figure 1: Moodle & Plugin dependency graph

3.2.2 Environment Setup

After figuring out the project, the work turned to Docker: software that packages a program together with everything it needs into isolated ‘containers’, so that the same setup runs identically on any machine [10]. Due to a lack of knowledge, Docker had to be learned for the first time; however this allowed a much faster setup and an automatically repeatable environment. This allowed Moodle to run, but not CodeRunner yet, because the in-development copies of the plugins had to be connected into the running Moodle (using Docker’s bind mounts, which share a folder on the host machine with a container). This did not work as planned because the strict permissions of the macOS laptop used for development stopped the shared folders from working properly. Once this was realized, development switched to a desktop machine running Linux, which fixed the issue. The project still did not work because Moodle’s strict security policy blocks the web requests its components use to talk to one another (cURL requests), even local ones between Docker containers. Allowing these was required because one of the CodeRunner dependencies is Jobeinbox, the code-running server described above. This was not obvious at the time, and many hours were spent trying to figure out what was wrong with the Docker setup.

3.2.3 Problem Identification

The project was analysed by researching what issues CodeRunner has: searching the public repository for posted issues, fielding a survey, interviewing lecturers, requesting fellow students’ opinions, and including first-hand observations from using the system. It quickly became clear that what had been presumed to be a lecturer problem was actually a software problem. User design books like Don Norman’s *The Design of Everyday Things* [14] show that user error often isn’t the user’s fault. In the case of CodeRunner this was and still is absolutely correct. CodeRunner is one of the many software tools that will forever need to evolve due to the growing requirements of education [17].

3.2.4 Review of the Author Page

The existing design for both student facing and author facing pages was reviewed against the design literature introduced in Section 2: Don Norman [14] and *Don’t Make Me Think* by Steve Krug [7] for the interfaces themselves, and Google’s developer documentation style guide [2] as the yardstick for the written documentation.

The author page explains the frustration felt towards lecturers: the user experience provides a full suite of tools with no instructions on how to use them properly. A long single page document can be

found on the CodeRunner documentation [1], and it is extremely well-written and complete; however, it falls short of Google’s guidance, which asks writers to organise content around the reader’s task, to keep pages short and scannable, and to avoid explaining more than the task at hand requires. The CodeRunner manual does the opposite on each count: it is organised around the system’s features, lives on one enormous page, and explains far more than the basic user needs. It is as though IKEA sent you a wardrobe with instructions on how to build every single type of wardrobe in their catalog and it is up to you to figure out which instructions you need to follow. This assessment directly shaped the documentation requirement (NFR3) and the task-focused documentation page later built to satisfy it.

Customisation		☒ Customise ☐ Template debugging
Answer box		Rows 18
Submit buttons		Precheck Disabled ☐ Hide check
Stop button		Never available
Feedback		Force show
Marking		☒ All-or-nothing grading Penalty regime: 0

Figure 2: The CodeRunner author page’s question-option controls, showing the per-label help buttons and the inconsistent label alignment discussed in this section.

The author page (Figure 2) is rough around the edges, and the burrs show on test cases: you can only add 3 test cases per request, no more no less. You also can’t delete the excess test cases. The user interface as a whole is also very dense, jumbled and indecisive; some labels appear above their selectors, some appear to the left of them. Checkboxes scattered across the page, misaligned with their labels also. The corners of the test case boxes are also completely square, unlike every other box on the page, which are all rounded. Every help button beside every major label does nothing aside from opening one large column of text that explains what the feature does, but it disappears

the moment you click elsewhere, and the text is difficult to read properly because it can overflow due to length (Figure 3).

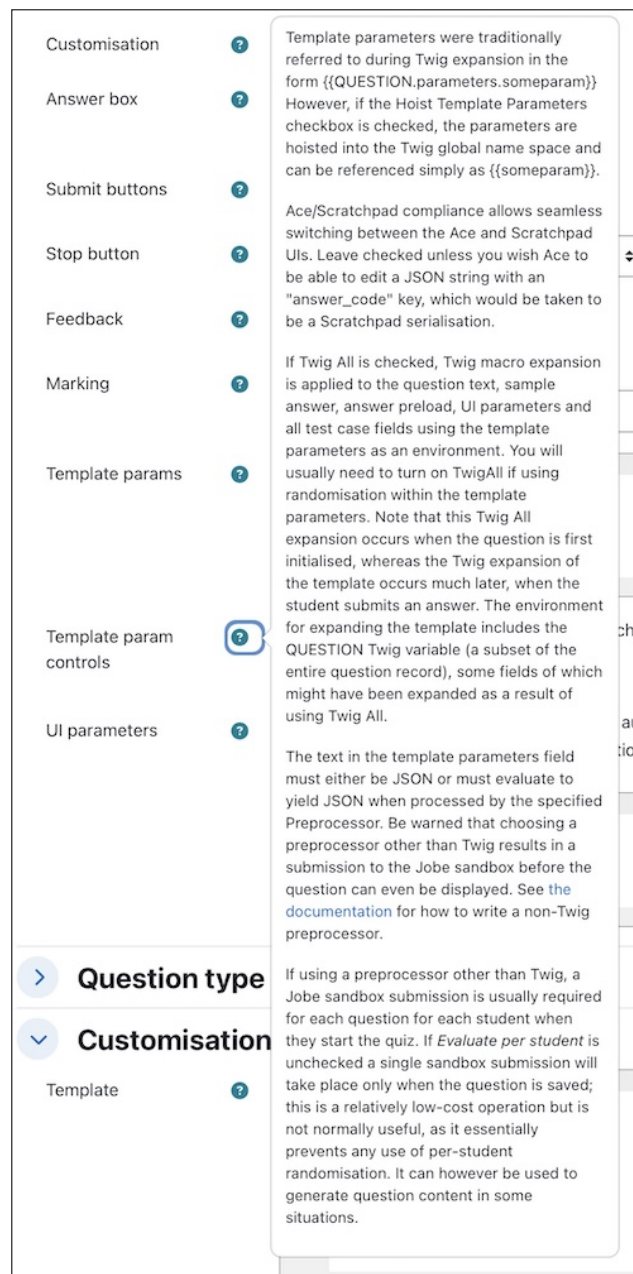


Figure 3: Author page showing the almost overflowing help hint discussed.

When the Precheck option is enabled, it appears in the label column of the test case instead of the input column to its right, and the padding of the row above separates it visually from the test case box it belongs to, appearing disjointed. A Lighthouse audit (an automated page-quality check built into the Chrome browser) scores this page at only 93% for accessibility, a measure of how

usable a page is for people relying on assistive tools such as screen readers, instead of the preferable 100%: several of the input controls in CodeRunner lack ARIA labels (invisible text that a screen reader speaks aloud), as does the Ace Editor, the code-editing box embedded in the page. Despite the author page being very technically heavy, there are very few self-explanatory options; the help buttons make some of them self-evident, but they often fail to explain how the question lifecycle really works [7].

3.2.5 Review of the Student Page

The student-facing question answer page is polished and complete. This page is quite good when the author writes the questions correctly; for example, a bad question would be where the question description is too massive to view both the description and the answer box at once. Another example is when the author chooses to use hidden test cases (tests whose details are concealed from the student, so that answers cannot simply be written to pass them): this is good for the author but it does not help the student at all. Often an author will have many hidden test cases and it is unknown which one the student failed, especially under complex questions. Inspection of the code showed that a written answer in the page is only saved automatically every 60 seconds, in the background; while this is acceptable, the user might benefit from an affordance (in Don Norman's sense [14], a control that visibly invites its use) which allows them to save manually.

3.2.6 Survey Analysis

Furthermore, pursuing a survey, designed with reference to established principles of survey research [6], yielded further requirements and analysis. A list of comments and suggestions follows:

- 'I don't like that hidden test cases don't tell the user where they went wrong', paraphrased, in reference to student-facing page, the system currently does not tell the student why they failed a hidden test case within a question.
- 'A hint option that deducts marks if used' in reference to the student-facing page, the system should allow the user to get hints in exchange for a reduced mark. This option is very similar to giving hints on hidden test cases failed instead of silently erroring out.
- 'An LLM to check whether the answer is close enough may be beneficial as CodeRunner often expects unrealistically exact inputs/outputs.' in reference to the grading methods available.
- 'An attached command terminal for outputs would be useful for interactive prototyping.' in

reference to the student-facing page, it shall allow the user to run custom commands and tests.

- ‘It would be nice if it had LSP support.’ in reference to the editor, the idea is to extend the editor’s ability to flag errors and runtime errors prior to the answer being checked.
- ‘Provide commands such as the gcc line which could be copied into a clipboard’ paraphrased, in reference to the student-facing page, the question should provide a codebox which allows the user to click a copy button instead of selecting and copying it manually.

In the order listed above, the following discusses the viability and benefit-to-cost ratio of each suggestion.

Providing a hint button is an idea very similar to allowing the user to get more information out of hidden test cases; the latter is arguably the bigger problem because in most cases the questions the lecturers write are well detailed and more than enough. The issue with hidden test cases is that often the student can be left guessing what is wrong instead of getting a detailed answer as to why they are failing, which costs them even more penalty if they guess wrong. In the real world, a developer can usually find a reason as to why something in their code isn’t working correctly.

An LLM (large language model, the technology behind tools such as ChatGPT) addition to the question grading options is an interesting suggestion because it is a genuinely good solution in low-complexity questions, but in higher complexity questions this method can give different grades to the same answer across two different cases. It is therefore unreliable, rolling the dice with each grade, and it leaves the author susceptible to scrutiny; recent benchmarking still places even state-of-the-art models behind human tutors at grading tasks [16]. The level of complexity to implement such a feature isn’t the problem because you can create something similar to Jobe or extend Jobe, however there is almost no university with enough money or hardware to run LLMs for grading at full capacity.

An interactive terminal included in a question is an appealing idea, however the complexity of implementing such an element is far beyond the scope of the project. To implement this, the system requires a host which is able to expose 100 or more simultaneous containers for use, as well as websocket connections. It would most likely act like SSH, however written in JS, the problem with this is that rewriting SSH in JS would prove incredibly time consuming and a security nightmare.

The Language Server Protocol (LSP) implemented into the Ace editor is a great idea; several contributors have managed to write a package, `ace-linters`, which does this exact thing. The trouble would most likely be controlling how to get fine control over the linter via PHP when the

author does not want this feature enabled.

Providing a copy button inside codeblocks would be a good idea and possesses a high benefit-to-cost ratio.

3.3 Requirements

3.3.1 Development Environment Requirements

- DE1** The environment must run Moodle, CodeRunner and its dependencies in isolation from the host machine. (Analysis)
- DE2** The environment must be startable with a single command and produce a consistent, repeatable setup. (Analysis)
- DE3** The environment must allow CodeRunner to be tested for regressions. (Analysis)

3.3.2 System Functional Requirements

- FR1** The author page must allow the author to delete test cases. (Analysis)
- FR2** The author page must allow the author to add exactly the number of test cases they want. (Analysis)
- FR3** The author page must keep help content visible until the author dismisses it. (Analysis)
- FR4** The author page must render the Precheck option in its correct column, attached to its test case. (Analysis)
- FR5** The author page must provide an option to give hints for hidden test cases. (Survey)
- FR6** The student question page must allow the student to save their current work on demand. (Analysis)
- FR7** The student question page must allow the student to manage the layout of the question. (GitHub Issue)
- FR8** The student question page must allow the student to reclaim unused margin space to enlarge the question content area. (Analysis)
- FR9** The student question page must provide a copy button on code blocks within the question text. (Survey)

3.3.3 System Non-Functional Requirements

- NFR1** The author page shall be visually consistent: uniform corner rounding, and labels and checkboxes aligned with their controls. (Analysis)
- NFR2** The author page shall score at least 97% on a Lighthouse accessibility audit, the maximum attainable given its third-party components. (Analysis)
- NFR3** The documentation shall be effective and self-evident: task-focused and reachable directly from the question editing form. (Analysis)

The remaining survey suggestions (LSP support in the editor, an interactive terminal, and LLM-based grading) were considered and excluded from the requirements as out of scope; the reasoning is discussed in the Survey Analysis above.

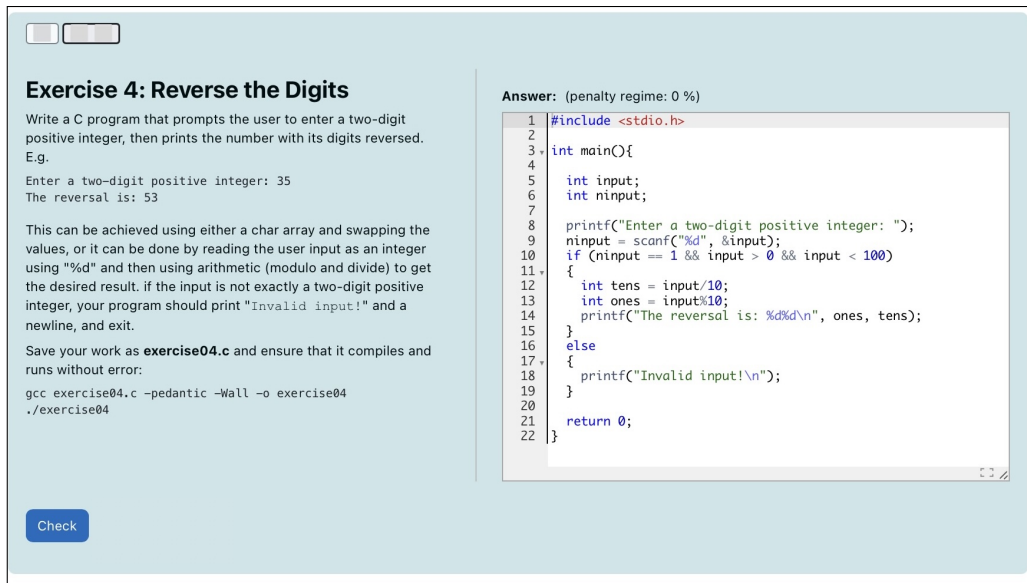
3.4 Design

The elements and layout were designed using in-browser prototyping in place of wireframes. This method seemed best because the environment contained the most information, and recreating a wireframe within a graphic design tool seemed slower than using the DevTools environment. The other benefit was that CSS rules could be tested live and rapidly verified to work as intended. For each feature a layout and style were drafted, the design shown to peers and the supervisor for review, and only then committed to CSS and JS files.

3.4.1 Interface Design

The following interface designs for each element were tested via a survey and opinions requested over email.

The layout switcher found on the student-facing page is designed with a toggle to allow the student to choose between two interface layouts (Figure 4). It gives the user control over how the question is presented, which is usually a net positive for user experience.



Exercise 4: Reverse the Digits

Write a C program that prompts the user to enter a two-digit positive integer, then prints the number with its digits reversed. E.g.

Enter a two-digit positive integer: 35
The reversal is: 53

This can be achieved using either a char array and swapping the values, or it can be done by reading the user input as an integer using "%d" and then using arithmetic (modulo and divide) to get the desired result. If the input is not exactly a two-digit positive integer, your program should print "Invalid input!" and a newline, and exit.

Save your work as **exercise04.c** and ensure that it compiles and runs without error:

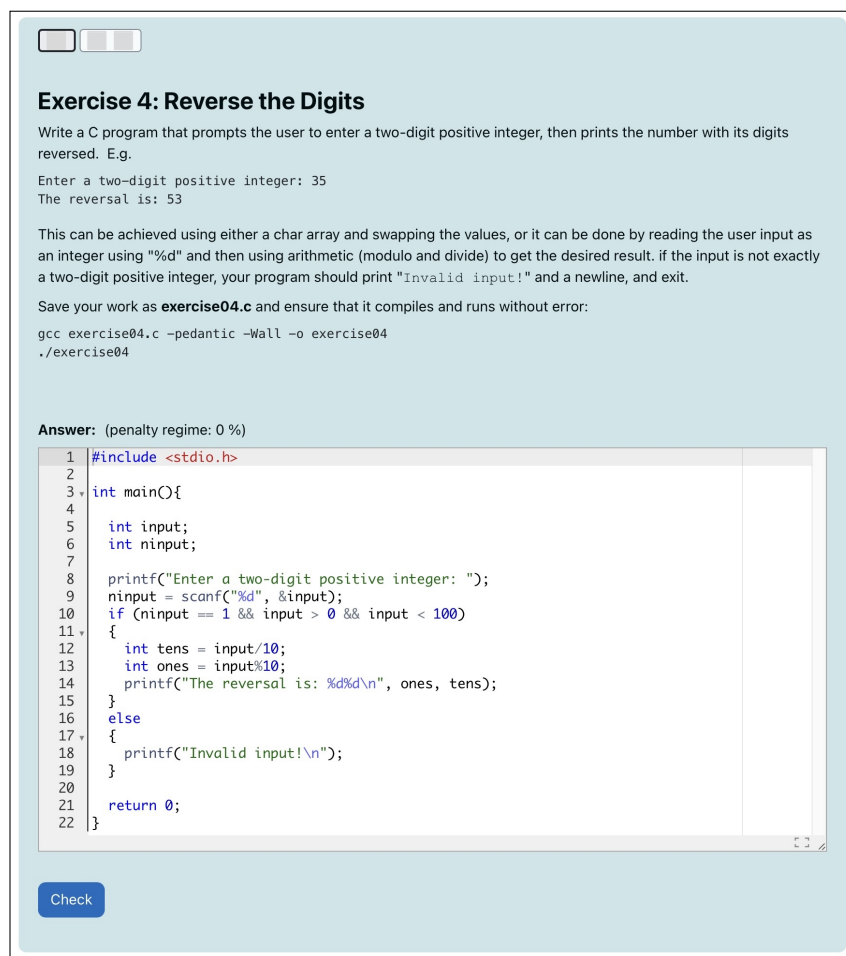
```
gcc exercise04.c -pedantic -Wall -o exercise04
./exercise04
```

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2
3 int main(){
4
5     int input;
6     int ninput;
7
8     printf("Enter a two-digit positive integer: ");
9     ninput = scanf("%d", &input);
10    if (ninput == 1 && input > 0 && input < 100)
11    {
12        int tens = input/10;
13        int ones = input%10;
14        printf("The reversal is: %d%d\n", ones, tens);
15    }
16    else
17    {
18        printf("Invalid input!\n");
19    }
20
21    return 0;
22 }
```

Check

(a) Side-by-side view



Exercise 4: Reverse the Digits

Write a C program that prompts the user to enter a two-digit positive integer, then prints the number with its digits reversed. E.g.

Enter a two-digit positive integer: 35
The reversal is: 53

This can be achieved using either a char array and swapping the values, or it can be done by reading the user input as an integer using "%d" and then using arithmetic (modulo and divide) to get the desired result. If the input is not exactly a two-digit positive integer, your program should print "Invalid input!" and a newline, and exit.

Save your work as **exercise04.c** and ensure that it compiles and runs without error:

```
gcc exercise04.c -pedantic -Wall -o exercise04
./exercise04
```

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2
3 int main(){
4
5     int input;
6     int ninput;
7
8     printf("Enter a two-digit positive integer: ");
9     ninput = scanf("%d", &input);
10    if (ninput == 1 && input > 0 && input < 100)
11    {
12        int tens = input/10;
13        int ones = input%10;
14        printf("The reversal is: %d%d\n", ones, tens);
15    }
16    else
17    {
18        printf("Invalid input!\n");
19    }
20
21    return 0;
22 }
```

Check

(b) Stacked view

Figure 4: The two student-facing question layouts offered by the layout switcher.

The Draft button found on the student-facing page was designed with the Bootstrap CSS framework in mind: Moodle itself uses Bootstrap at its core to style their interface, which allows existing button styles to be reused. With respect to the other existing buttons, the Draft button is styled as the lowest priority, the least attention-grabbing; the gray outlined button is barely noticeable until it is looked for: discoverable but not offensive to the eyes (Figure 5).

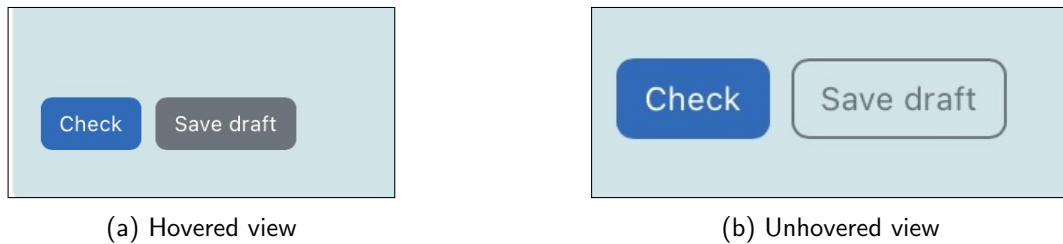


Figure 5: The Save Draft button, hovered and unhovered.

The test cases found on the author page had their styles updated, and a new button was added for deleting test cases. This button also requires the user to confirm their choice using an alert dialog to prevent accidental work loss.

The info panel on the student-facing page was slightly reworked by the addition of a fold/float button. This button was originally designed to completely hide the info panel to allow the page to reduce its margins therefore allow the main content to consume more space for improved readability; however, experimenting with the panel revealed it already supported a different variant of this in the mobile version of the student-facing page. The in-place alternate style is leveraged by allowing the user to use a button to manually set the info panel into mobile mode.

The documentation page added by this project required almost zero interface design because of how the system is designed: the input for this page is entirely Markdown (MD), which lets the PHP MD parser take care of the element and CSS styling. The output is then rendered as HTML onto the page. The input is also a part of the interface because it is written in Markdown, which makes writing up-to-date documentation far more accessible and readable. To quote a survey response: 'Currently the best way to learn CodeRunner is to have someone that knows it explain it to you.' Making the documentation accessible and easy to update means that it has the potential to be extended in the future by other contributors and CodeRunner question authors who are experts in it.

3.4.2 System Design

The following diagram describes the Docker topology designed for the development part of this project:

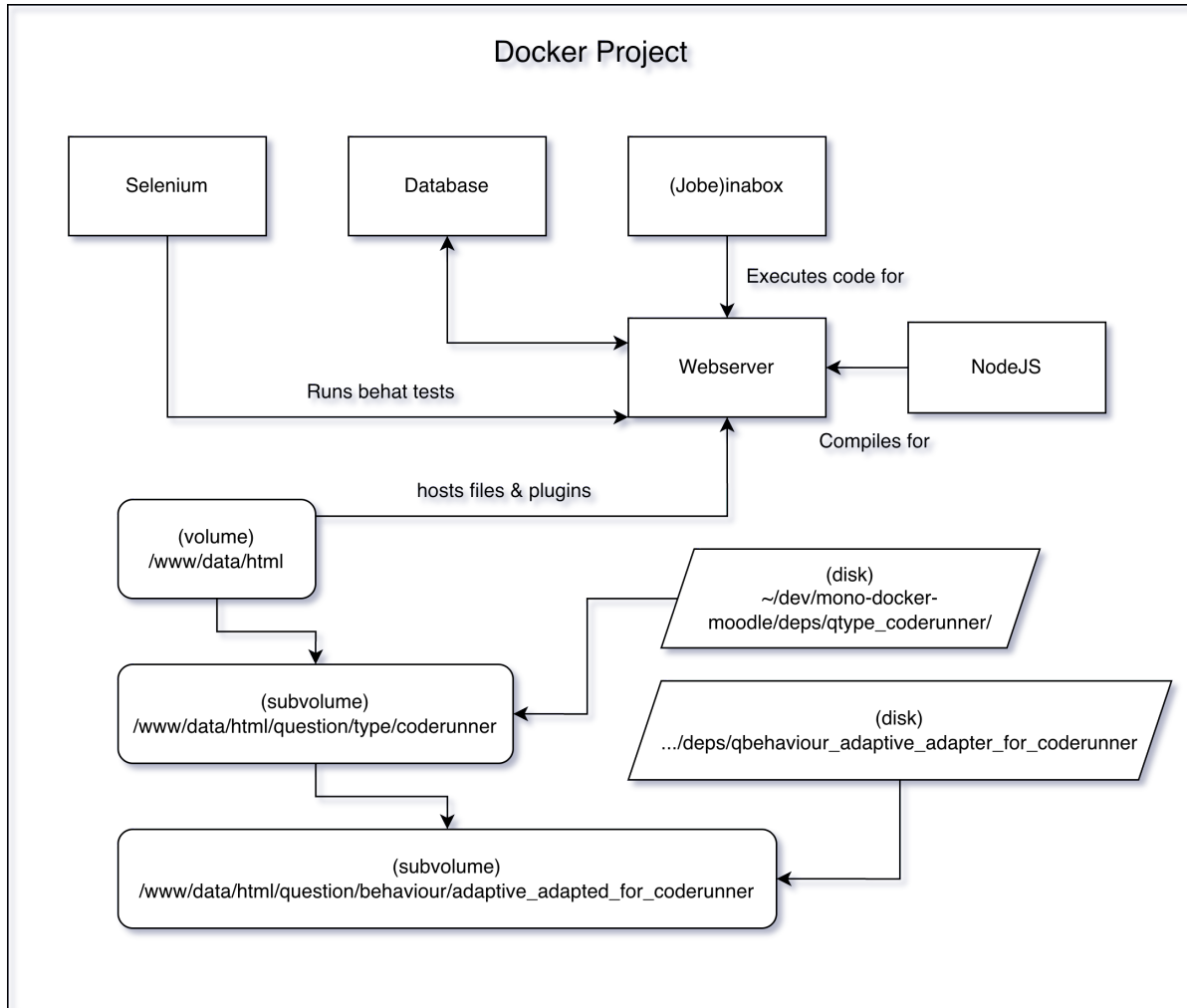


Figure 6: Docker architecture

The following diagram is in direct reference to the analysis and describes the sequence of how CodeRunner and Moodle interact to fulfil a question result.

To save the layout state and question info hidden state, a `sessionStorage` object is used. This allows the page to remember what layout each question was in, sparing the user the cognitive load of recognising what happened to their questions. The data structure avoids polluting `sessionStorage` with a separate entry per question by keeping all question states under a single key. The data structure in TypeScript goes as follows:

Listing 1: TypeScript definition for Layout & Question Info state

```
1 type QuestionState = {layout: string, infoCollapsed: boolean};
2 type QuestionID = string;
3 type QuestionStateTracker = Record<QuestionID, QuestionState>;
4 // Instead of a new sessionStorage entry per question,
5 // all question states are stored under one key.
6
7 const STORAGE_KEY = 'coderunner_layout';
8 let state: QuestionStateTracker =
9     JSON.parse(sessionStorage.getItem(STORAGE_KEY) || '{}');
10 let question = state[questionID];
11 let layout = question.layout;
12 let infoCollapsed = question.infoCollapsed;
```

In some circumstances where the default option is different from the user-chosen option, such as the alternate layout, the page may render with the default option (e.g. style rules), then a lazily loaded script changes it. This sequence happens within 500ms on a good machine, which appears janky and broken. To prevent this, a script was added to the main renderer for CodeRunner which is injected into the page and runs before the first paint, preventing the janky behaviour by overriding the defaults to be the user-chosen ones instead.

The test cases in general required a rework to support deleting test cases. Developing a delete test case button for the author page revealed that test cases were managed and accessed by a continuous index, which broke when deleting test cases in the middle of the array. The parts that interacted with the list of test cases were rewritten to access by key instead of array index. This introduced no regressions according to Behat, PHPUnit and manual testing, and allowed the delete test case button to work correctly.

The lack of a database design is a deliberate decision: `sessionStorage` was chosen because the changes live in the browser instead of the persistent database. This seemed the most appropriate and least time-costly method. Of all the many changes made, any touching the backend are exclusively stateless or required zero changes in database schema, therefore only browser data needed to be saved. From an end-user's perspective, such a feature would not be expected to save the layout across session resets.

4 Product

4.1 Implementation

The implementation of this project uses PHP for Moodle-side changes, JavaScript for Moodle AMD modules (compiled with Grunt via the NodeJS container), CSS for styling, Markdown for documentation content, shell scripts for development workflow.

4.1.1 Development Environment

The development environment uses Docker [10] (Figure 6), which deploys several containers including Selenium for testing and NodeJS for compiling. The environment maintains a virtual volume containing the entirety of Moodle and its files, then Docker mounts the files for CodeRunner and its dependency plugin. The docker environment can be used by executing shell scripts; a couple of scripts were implemented for the purpose of automating environment variable assignment and executing complex commands for Behat and compiling AMD modules in one go.

```
$ docker volume ls -q --filter "name=moodle-docker" | xargs docker volume rm
```

This setup meets **DE1** because the Docker container is isolated and the data produced within is contained. Deleting the hosted volumes within Docker allows starting over without config file hunting across the host. **DE2** is met using a shell script 'quick-start.sh' which loads the environment variables required to start and executes the docker-compose bin to init and create the environment. **DE3** is fulfilled by the Selenium container, the Selenium image loads a headless chrome browser controlled by Behaviour Driven Development tests (BDD) [15] using Behat.

4.1.2 Test Case Management

FR1 was met by leveraging an existing capability of Moodle's forms library (formslib). The library's `repeat_elements()` helper, which CodeRunner already used to render one group of fields per test case, supports author-facing addition of rows but offers row deletion only as an opt-in: its final parameter nominates a button within each repeated group as that row's delete control. A submit button named `deletetestcase` was therefore added to the repeated group and its name passed to `repeat_elements()`. Moodle registers each `deletetestcase[i]` as a no-submit button: clicking it reloads the form without validating or saving, and on the rebuild formslib omits the deleted row and

records the deletion in a hidden field so that it survives further reloads; the deletion only becomes permanent when the question is saved. The button's visible label is not derived from its name; it is looked up with `get_string('deletetestcase', 'qtype_coderunner')` from the plugin's language file, to which the new strings were added, keeping the label translatable like every other string in the plugin.

To guard against accidental deletion, a small script was attached to each delete button that asks for confirmation via the browser's native dialog. The script does not replace the button's behaviour: it merely intercepts the click and cancels the submission when the author declines, so a confirmed click proceeds exactly as it would have without the script. This makes it a progressive enhancement [9]: were JavaScript unavailable, deletion would still work, just without the prompt.

Finally, the delete button alone was not enough, because CodeRunner accessed test cases by contiguous index, e.g. `[0, 1, 2, 3, 4]`. If the author deletes test case 1, the submitted arrays instead have the keys `[0, 2, 3, 4]`, a hole in the sequence. Any loop that assumed a dense range would both read the missing index and silently drop the last row, so every such loop had to change to iterate over the keys actually present. Four places were affected: the form-building loop that attaches help buttons and per-row rules to each test case, the validation of Twig-rendered fields, the validation that counts non-empty tests, and the function that copies test cases from the form into the database on save. The validation of non-empty tests is the most interesting case: the fields `testcode`, `stdin` and `expected` are submitted as three parallel arrays sharing row keys, and the old code walked them with a single counter, assuming they were equal in length with no holes. The new code instead takes the union of the three arrays (PHP's `+` operator on arrays, which keeps every key appearing in any operand) and iterates over the `array_keys()` of the result, so a row is validated if any one of its three fields is present, whatever its key happens to be.

FR2 was only partially met, and the compromise was deliberate. Upstream, a new question opens with five blank test cases, and the add button appends three more per click, an amount CodeRunner inherited from a Moodle core constant rather than choosing itself. The core constant was first replaced with plugin-owned ones set to one, which met the requirement exactly: a new question opened with a single blank test case and each click added exactly one more. Once the Behat suite was runnable inside the container, however, this turned out to break several of the upstream acceptance tests, which fill a freshly opened form up to its third test case (`id_testcode_2`) and so assume at least three blanks exist before any button is clicked. Rather than rewriting upstream test fixtures for what is largely a cosmetic gain, both constants were reset to three, the smallest initial

count for which the unmodified upstream suite still passes. The result improves on upstream (three initial blanks rather than five) but is not exact; the ideal solution, letting the author choose how many rows each click adds, was judged too low in value relative to its cost in test maintenance and deferred. In practice the gap is also narrowed by FR1: surplus blank rows can now simply be deleted.

4.1.3 Precheck Rendering Fix

FR4 was met through in-browser analysis followed by a deliberately small fix. When the Precheck option on the author page is set to 'Selected', each test case gains an extra div holding its test-type selector, and this div was rendering outside the test case's column flow. Inspecting the live page revealed why: the stylesheet lays each test case row out as a flex container, but the script that shows and hides the selector (necessarily client-side, since the div must react to the Precheck setting without a page reload) re-showed it with an inline `display: block`. Inline styles override the stylesheet, so every toggle dropped the div out of the flex layout. The fix was a single word in the `authorform.js` module: re-show the div with `display: flex` instead. The size of the change is out of all proportion to the analysis required to locate it, which is arguably the mark of a well-aimed fix rather than a small effort.

With the div rendering correctly, one styling wrinkle remained. The selector sat at the bottom of each test case card, where the card's rounded-corner styling belongs to the last visible element, but whether the selector is visible depends on the Precheck setting. That left two options: reorder the form elements so the selector sits in the middle of the card, or dynamically pass the rounded-corner styling between the selector and the element above it as visibility changes. The reorder was chosen, a small server-side change to the form definition, accepting the minor cost that authors accustomed to the old layout must find the selector in its new position; since it moved past only a single element group, that cost is low. The alternative would have tied presentation logic to the Precheck state for a purely cosmetic outcome.

4.1.4 Save Draft Button

To meet **FR6**, a Save draft button was added to the controls beneath the answer box. Previously, a student's only explicit way to save was the Check button (a graded submission that can cost marks under the question's penalty regime [8]), so saving work-in-progress meant either paying that penalty or trusting the browser not to lose the page. The new button provides a zero-cost, on-demand persistence point: it stores the current answer and changes nothing about grading. It should be

noted that Moodle does periodically (every 60s) automatically save the answer box content. It is, in short, an affordance [14].

The change is small (28 lines across three files) and lives entirely in the companion behaviour plugin rather than the question type. This placement was deliberate. In Moodle's question architecture [13], the question type defines what a question is, while the behaviour governs how responses are processed; a save control is a submission concern, so it belongs to the behaviour. It also means any question using this behaviour gains the button, and the question type remains presentation-only.

The implementation exposes existing machinery rather than building new. The renderer emits a submit button whose name is generated by `get_behaviour_field_name('savedraft')`, Moodle's namespacing scheme that routes the click to the correct question's behaviour even when several questions share a quiz page; the construction mirrors the core `submit.button` renderer so the button participates in the question engine's usual JavaScript handling, and it is rendered inert on read-only review pages. The behaviour then declares `savedraft` as expected data, but only while the attempt is unfinished; the engine discards any submitted field a behaviour has not declared, so this single declaration is what makes the button function, and the guard ensures a closed attempt can never be drafted against. Notably, no processing code was required at all: because `savedraft` is not one of the engine's recognised grading actions, the submission falls through to the engine's default save path, which stores the response as a new step without grading or penalty. The feature's label is resolved from a new language string, keeping it translatable like the rest of the plugin.

4.1.5 Layout Switcher

To meet **FR7**, a per-question layout switcher was built offering two arrangements: the traditional stacked view, and a side-by-side view that places the question text and the answer box in adjacent columns. The server-side renderer was changed to wrap the two halves in dedicated `question_box` and `answer-box` containers (CSS can only arrange what the markup separates), and a new AMD module, `layoutswitcher.js`, is initialised for each CodeRunner question on the page. The module injects a pair of toggle buttons and implements the modes as a single CSS class toggled on the question's outer div: the JavaScript only flips state, while the layout rules themselves live in the stylesheet.

In the side-by-side mode, a draggable divider separates the two boxes. Dragging recomputes the boxes' widths as ratios, written to their `flex` properties, so a chosen split survives later window

resizes proportionally instead of pinning either side to a pixel width; both sides are clamped to a minimum width (150 pixels, or half the available space when narrower) so neither box can be dragged to nothing or past the other. One subtlety is worth recording: switching modes explicitly clears any inline `flex` values a previous drag left behind, because inline styles would otherwise override the stylesheet's rules indefinitely, the same class of fault as the Precheck rendering bug above.

The chosen layout persists in `sessionStorage` under a single key, recorded per question, so two questions on the same quiz page can hold different layouts. The stored entry began life as a bare layout string and later grew into a small object shared with the info panel state of the next section; a migration path still accepts the legacy form. `sessionStorage` was a deliberate choice: it needs no server round-trip and no database schema change, and it holds nothing personal, only a layout preference keyed by question id, a point the Security section returns to.

Finally, the saved layout is applied before the page paints. Because the AMD module loads asynchronously, a question saved as side-by-side would otherwise render stacked and visibly jump when the module initialised, a layout shift [11]: the jank problem discussed in Section 3. The renderer therefore emits a small inline script ahead of the question markup that reads `sessionStorage` and applies the layout class before first paint, eliminating the flash; it is wrapped in a `try/catch` that falls back to the stacked default wherever `sessionStorage` is unavailable. Throughout the module, every DOM lookup is guarded, so a question rendered without the expected markup silently receives no controls; and the entire feature is client-side, with no question-processing code touched, so it carries no grading or data-storage risk.

4.1.6 Info Panel Collapse

To meet **FR8**, the student is given a way to reclaim the width that a question page holds in reserve. Two separate things waste horizontal space on a CodeRunner question, and both matter more here than for ordinary question types, because code wants room: Moodle floats a fixed-width info panel (question number, status, marks) beside every question, indenting all of its content; and the Boost theme caps the width of the page content as a whole, leaving dead margins on either side. A small toggle button added to each question's info panel reclaims both at once. Collapsing removes the panel's float, so it stops reserving its column and reflows as an ordinary block while the question content expands beneath it, and simultaneously raises the theme's page-width cap so the whole content area widens into the margins.

The feature went through one design iteration worth recording. The first version hid the panel entirely (shrinking it to the size of the toggle button and hiding its children), which reclaimed the column at the cost of the question number and marks disappearing with it. The final version reflows rather than hides: no information is lost; it merely stops costing a dedicated column. The distinction between removing content and removing the space reserved for content is what the requirement was really asking for.

Two details deserve mention. First, the page-width half of the feature must coordinate across questions: a quiz page can hold several questions, all sharing one page-width cap. The module therefore keeps a page-level set of every question whose panel is currently collapsed; the cap is lifted when the first panel collapses and restored only when the set is empty again, since restoring it any sooner would shrink the page out from under another question still relying on the extra width. Second, the page-level step is deliberately theme-tolerant: it targets a wrapper element specific to Boost-family themes, and on any theme without that markup it is a guarded no-op; the panel itself still collapses normally, so the feature degrades to its within-question half rather than breaking.

The collapsed state persists per question in the same `sessionStorage` record as the layout choice (this is the change that grew the stored entry from a bare layout string into the small object described in the previous section), and the renderer's pre-paint script applies it before first paint, giving the collapse the same protection against the flash-and-jump behaviour as the layout itself. On narrow (mobile-width) screens the toggle is hidden entirely: the info panel already stacks above the question there, so there is no column to reclaim.

4.1.7 Accessibility

To meet **NFR2**, the author page was audited with Chrome's Lighthouse tool, whose accessibility checks derive from the Web Content Accessibility Guidelines [19], and the findings were worked through. The page initially scored 93%. The largest issue was that several of the page's selects are rendered by Moodle as grouped controls (the question type, give-up and feedback groups) where the visible label belongs to the group rather than to the select inside it, leaving the selects with no accessible name. This was fixed with `aria-labelledby` attributes [18] pointing each select at the id of its group's existing label element. `aria-labelledby` was chosen over `aria-label` on purpose: it reuses the visible text instead of duplicating it, so a future edit to the label cannot silently desynchronise the accessible name. This took the audit score to 97%, and the remaining three percent lies in TinyMCE, Moodle's bundled rich-text editor, which the plugin cannot modify;

this measured ceiling is where the requirement's figure of "97%, the maximum attainable given third-party components" comes from.

The new controls this project introduced were held to the same standard. The layout switcher's buttons are icon-only (two squares against one), meaningless to a screen reader, so both carry descriptive `aria-labels`; the info panel toggle does the same, updating its label to match its state. The same buttons also gained visual contrast: they originally sat at low opacity with pale borders, making the active state nearly indistinguishable, and were reworked with solid backgrounds, darker borders and an unambiguous active style, so state is perceivable without relying on subtle opacity differences.

One piece of accessibility work remains unfinished, and is reported as such. When a student runs their code, the test results should be announced to a screen reader when they arrive; `aria-live` [18] was added to the results container and the results message given an addressable id in preparation. It does not yet work, and the cause is known: the results div is replaced wholesale on submission rather than mutated in place, so the live-region registration is destroyed along with the element it was attached to. Fixing this properly means restructuring how the renderer swaps in results, which is scoped as future work.

4.1.8 Documentation Page

To meet **NFR3**, a documentation page was added to the plugin itself. Upstream CodeRunner's documentation lives on an external site, which the editor's help strings link out to; an author who gets stuck mid-task must therefore leave Moodle and search a general-purpose manual. The new page instead serves task-focused documentation from inside the plugin, following the task-oriented, scannable structure recommended for developer documentation [2]: a new script, `docs.php`, renders Markdown pages within the normal Moodle page chrome, so the documentation is styled and navigated like part of the site the author is already working in.

The mechanics were chosen for simplicity of authorship. Content is plain Markdown in a directory inside the plugin, rendered with Moodle's bundled `MarkdownExtra` parser (introducing no new dependency), and the directory is scanned recursively for pages, so adding documentation means adding a file, with no registration step. One rendering defect had to be fixed along the way: `MarkdownExtra` emits headings without `id` attributes, so in-page links to sections silently did nothing. A custom `header_id_func` was supplied that slugifies each heading into an anchor id, restoring in-

page navigation. This in turn surfaced a PHP 8.1 deprecation notice inside the parser itself, which is silenced narrowly around the single transform call, with a comment recording why and a note to contribute a proper fix upstream.

Because `docs.php` turns a request parameter into a file path, it is the one place among the project's changes that handles a classically dangerous input, the path traversal weakness [12]. The parameter is first cleaned by Moodle's `optional_param()` with `PARAM_PATH`, then resolved with `realpath()` and required to lie inside the documentation directory; traversal attempts and nonexistent pages alike raise a Moodle exception rather than reaching the filesystem. The script was also written to be testable: it declares its globals explicitly so PHPUnit can include it from within a test, and its unit tests attack exactly this path-containment defence, as the Verification section describes. The content written for the page comprises a revised editor-usage guide and a new question-templating guide.

4.1.9 Security and Personal Data

4.2 Verification & Validation

4.2.1 Verification

Automated verification rests on the two test harnesses of the development environment, each runnable with a single script. For the documentation page, PHPUnit tests were written. `docs.php` is a plain script rather than a class, so there is nothing to call directly: the tests instead fake the request by setting `$_GET` (which `optional_param()` reads) and include the script inside an output buffer, capturing whatever it renders. Three tests cover the page: a real document renders; a nonexistent page raises an exception; and a traversal attempt requesting `../../version.php` is rejected. That last target was chosen with care: it is a real file outside the documentation directory, so the test cannot be satisfied by a naive existence check and instead attacks the path-containment defence directly.

For regression testing, the plugin's upstream Behat suite was run: browser-driven acceptance tests, executed against the Selenium container by `behat.sh`. Their evidential value comes from their provenance (tests not written by this project, exercising code it changed), so passing them is independent evidence that the test case indexing rewrite preserved existing behaviour. The suite also demonstrably earned its keep: it exposed that the one-blank test case defaults broke upstream scenarios, forcing the reset to three described under FR2. That is a concrete regression caught by

the harness before any user saw it, and it justifies the effort spent making the suite runnable (DE3).

The remaining evidence is measured or manual. Lighthouse audits provide the quantitative check for NFR2, with the before and after scores (93% and 97%) recorded at the time of the change. The user-interface features (the layout switcher, the info panel collapse, the save draft button and the precheck fix) were verified manually, both directly and through the closed feedback loop of peers and supervisor described in Section 3. The limitation is stated plainly: automated coverage is concentrated on the documentation page and the upstream suite, and the new UI behaviour has no automated tests of its own.

4.2.2 Validation

Validation against the requirements is summarised in the traceability table (Table 1) [3], which traces each requirement from Section 3 to its implementation, the evidence that verified it, and its status. The statuses are kept honest rather than uniform: most requirements are met, FR2 is met partially with the trade-off argued in its implementation section, and two were deferred.

The deferred requirements were scoped out deliberately rather than silently dropped. FR5 (hints for hidden test cases) and FR9 (a copy button on code blocks) both originated in the user survey, and neither is abandoned: the Results section reframes the hint mechanism as the broader question of how much information hidden test cases should reveal, and identifies the copy button as future work with a high benefit-to-cost ratio. Deferring them was a scoping decision made against the time remaining, and the reasoning is recorded where the requirements came from, in the survey analysis, rather than asserted here.

Table 1: Requirements traceability: each requirement from Section 3, its implementation, and its verification evidence.

ID	Requirement	Implementation	Verified by	Status
<i>Development environment</i>				
DE1	Environment isolated from host	Docker compose stack	Manual	Met
DE2	Single command, repeatable setup	quick-start / compose scripts	Manual	Met
DE3	Regression testable	phpunit.sh, behat.sh	PHPUnit, Behat	Met
<i>Functional requirements</i>				
FR1	Delete test cases	repeat_elements delete button	Behat, manual	Met
FR2	Add exact number of test cases	NUM_TESTCASES 3/3	Behat, manual	Partial
FR3	Help content stays visible			
FR4	Precheck renders in correct column	authorform.js fix	Manual	Met
FR5	Hints for hidden test cases	Not implemented	–	Deferred
FR6	Save work on demand	Save Draft button	Manual	Met
FR7	Manage question layout	Layout switcher	Manual	Met
FR8	Reclaim margin space	Info panel collapse	Manual	Met
FR9	Copy button on code blocks	Not implemented	–	Deferred
<i>Non-functional requirements</i>				
NFR1	Visual consistency	styles.css rework	Peer review	Met
NFR2	Lighthouse accessibility $\geq 97\%$	ARIA labels	Lighthouse audit	Met
NFR3	Effective, reachable documentation	docs.php + content	PHPUnit, survey	Met

5 Results & Evaluation

5.1 Evaluation Process

During the iterative process of designing the interfaces, before they were finalised, the project supervisor helped make styling decisions.

In order to correctly evaluate the interface design, a survey was performed on participants specifically from University of Strathclyde in the Computer Science department. The demographic ranged from Students to Lecturers. No other personal information was collected and any personally identifiable information was redacted.

The survey as of *18 July 2026* settled on 24 participants.

Unfortunately, a functional assessment where end-users tangibly interact with the interfaces was not possible due to time constraints and issues surrounding what happens with the data collected such as GDPR and user consent. The data would be stored on personal drives, which would define the surveyor as liable.

It was decided it was best to use Qualtrics instead, where the survey contained 8 questions, varying by 2 less if you selected to be declare yourself a student. The removed questions were pertaining to the documentation requirement of this project, which meant they were not applicable to students.

5.2 Results of Evaluation

This Section includes a direct interpretation of the gathered data and evaluation processes. Figures 7, 8 and 9 summarise the responses to the three interface features shown to participants, with each question broken down by respondent type.

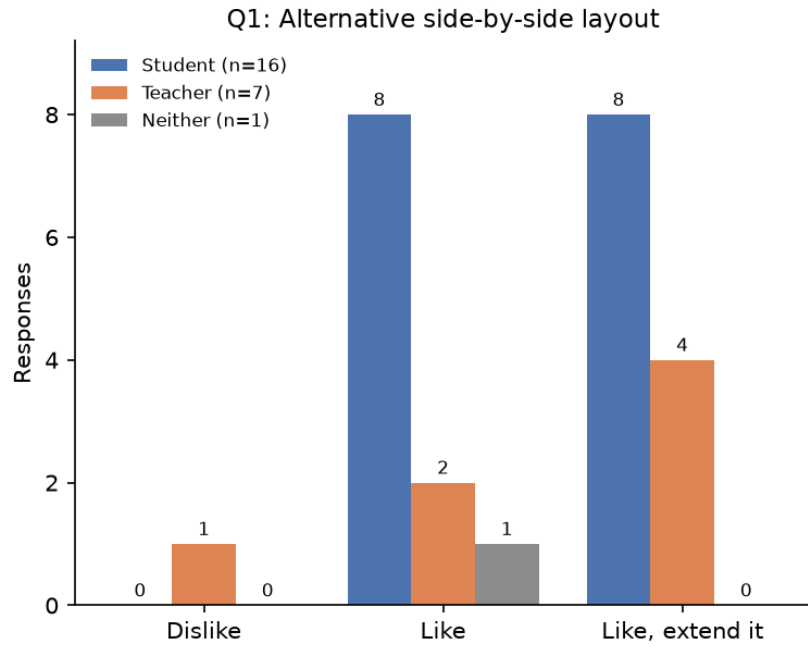


Figure 7: Survey responses to the alternative layout (Q1), broken down by respondent type.

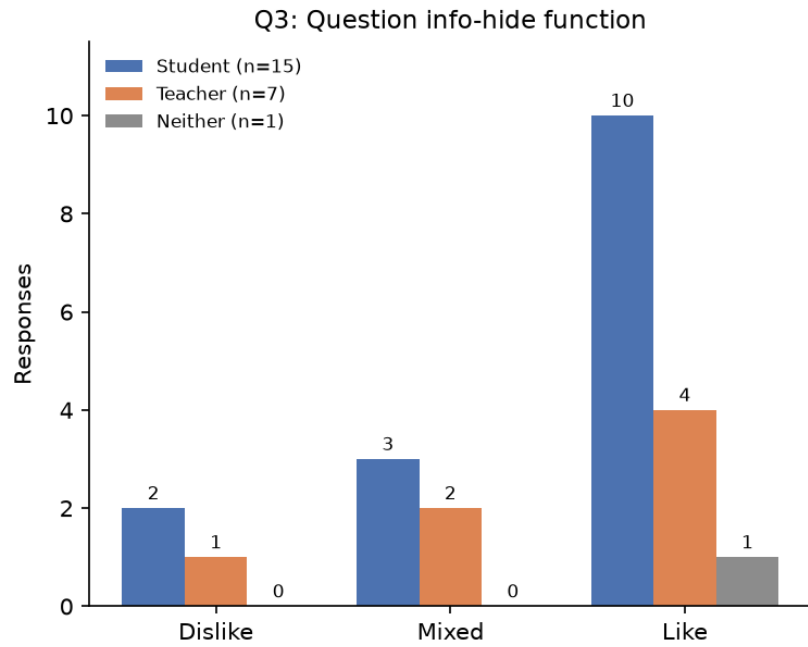


Figure 8: Survey responses to the info-hide function (Q3), broken down by respondent type.

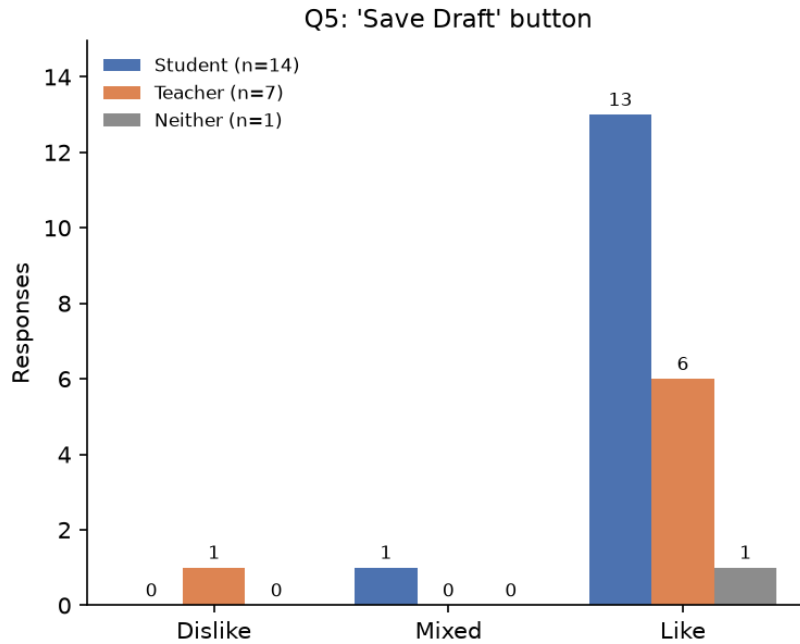


Figure 9: Survey responses to the Save Draft button (Q5), broken down by respondent type.

5.2.1 Reception of the Interface Changes

Reception of the three interface changes was strongly positive across both roles. The alternative side-by-side layout (Figure 7) received the broadest approval: of the 24 responses to the question, 23 (96%) were positive, and exactly half of all respondents chose the stronger option of liking it and wanting it implemented on other question types. That second figure is notable because it asks for more than FR7 ever specified; extending the layout switcher beyond CodeRunner questions is therefore recorded as future work. The single negative response came from a teacher.

The info-hide function (Figure 8) was the most divided of the three: 15 of 23 responses (65%) liked it as delivered, 5 (22%) judged it a good idea if improved, and 3 (13%) disliked it. This spread is consistent with the feature being the most novel of the three, since it changes how the page itself behaves rather than adding a discrete control. A majority of both students and teachers still approved, and the appetite for refinement points to polishing the interaction rather than removing it.

The Save Draft button (Figure 9) drew the highest approval rate: 20 of 22 responses (91%) liked it outright, with one response in each of the remaining categories. This is a reassuring result for a control that was styled to be deliberately unobtrusive, as it suggests the affordance was noticed and

valued regardless.

Taken together, the survey substantiates the claim that the delivered features were worth building: every feature achieved majority approval within both the student and the teacher group, and no feature was rejected by either group.

5.2.2 Documentation Findings

The two documentation questions were shown only to non-student respondents, as they concern authoring rather than answering. Seven teachers responded; five of the answers were substantive, and two were test entries which are excluded from the analysis.

On the current state of CodeRunner's documentation, the responses corroborate the assessment made in Section 3. Its quality was acknowledged while its form was criticised: 'good, but monolithic and hard to search' and 'okay, but not always the easiest to understand'. Two responses located the working knowledge outside the manual altogether, one noting that 'a lot of the useful stuff is within the forums and needs a bit of searching to find', and another reporting that they had not consulted the documentation since 2021, preferring to work from third-party examples. One respondent summarised the problem in the words quoted in the Design section: the best way to learn CodeRunner today is to have someone who already knows it explain it.

On the proposed alternative, documentation organised around the author form with contextual explanations, four of the five substantive responses were supportive, ranging from 'sounds good' to 'plenty of other areas use a similar approach so it seems like a no-brainer' and 'this would be useful to stop you needing to look it up'. The remaining response raised a considered governance concern rather than an objection: if authors can contribute to the documentation, someone must hold authority over debatable contributions, and clarity should be valued above concision. These points target the future contribution model rather than the documentation page delivered by this project, and they are recorded as considerations for any community-editable extension of it.

These findings validate the direction taken for NFR3: the criticisms of the existing manual match the deficiencies identified in the analysis, and the approach built in response was endorsed by four of the five substantive teacher responses.

6 Discussion & Reflection

6.1 Interpreting the Results

The survey results support the project's central bet: that a mature assessment tool's remaining problems are experience problems, not capability problems. All three interface changes achieved majority approval in every respondent group, and the strongest demand recorded was not for something new but for more of the same, with half of all respondents asking for the layout switcher to be extended to other question types. The most divided reception, for the info-hide function, belongs to the change that alters how the page itself behaves rather than adding a visible control; this suggests that features which move existing content need more explanation, or gentler defaults, than features which add something clearly optional.

The documentation findings carry a wider implication. The teachers surveyed did not criticise the quality of CodeRunner's manual, only its shape, and two of them reported working from forums and third-party examples rather than the documentation at all. This matches the analysis in Section 3 and suggests the real competition for official documentation is not better official documentation but informal knowledge scattered across colleagues and forums. Documentation that lives where the work happens, as built for NFR3, is an attempt to bring that knowledge back into the tool. The one governance concern raised, about who arbitrates contributions, is the correct next question and is addressed under future work.

6.2 Reflection

The clearest lesson is written in the project's own commit history: the test environment should have been completed before feature work began, not alongside it. The first behaviour-changing commit was made on 1 July and, among other things, reduced the number of blank test cases a new question opens with. The scripts that make the regression suites runnable only landed on 8 and 10 July, and the Selenium configuration that let the browser tests run reliably was fixed on 17 July. The first full run of the suite that day exposed that the 1 July change had been breaking upstream acceptance tests for over two weeks, and the fix was committed within hours of the harness working. Had DE3 been discharged first, a sixteen-day-old latent regression would instead have been a same-day correction. The requirement was identified early; its priority was misjudged.

A second lesson concerns commit discipline. Several commits bundle unrelated changes, such as accessibility labels arriving in the same commit as the new documentation page, or the test case

index rewrite sharing a commit with the delete button and a form reordering. This cost nothing at the time but made later work harder: writing this report, tracing when a regression entered, and understanding which change caused which effect all required untangling commits that should have been three commits each.

Finally, the history records a misjudgement about what counts as work. The commit message for the Precheck rendering fix reads 'this is probably the one legitimate fix I've made so far', written after hours of diagnosis produced a one-word change. With hindsight this undervalues exactly the work that mattered: the diagnosis was the fix, and the polish, styling and interface changes dismissed in that message were the substance of the project's requirements. Recognising analysis and refinement as legitimate engineering, rather than only new code, is a perspective this project taught.

6.3 Challenges

The environment consumed far more of the schedule than anticipated. Before any feature could be attempted, the project had to assemble four codebases into one repository, learn Docker from scratch, and diagnose two failures in sequence: macOS permissions silently breaking the shared plugin folders, which forced a switch to a Linux machine, and Moodle's security policy blocking the internal web requests its own components use, which cost many hours because the failure gave no useful signal. The lesson carried forward is that platform security policies should be the first suspect when two locally-run services refuse to talk.

The second sustained challenge was working inside a large, unfamiliar, live codebase. CodeRunner and Moodle's question framework are mature systems with their own idioms, and several tasks that appeared small required understanding those idioms first: the delete button was one parameter, but making it safe required discovering that the form library re-applies help buttons by row index and rewriting every loop that assumed test case indices were contiguous. Estimating work by the size of the visible change proved consistently misleading.

6.4 Limitations

The evaluation has clear limits. The survey settled on 24 participants, of whom only seven were teachers, and the documentation questions were shown to teachers alone, leaving five substantive responses after two test entries were excluded; conclusions drawn from that subset are indicative rather than firm. Participants judged the features from images and recordings rather than hands-on use, a constraint accepted for data protection reasons, so the results measure first impressions rather

than lived usage. On the product side, the new interface behaviour has no automated tests of its own, two requirements (FR5 and FR9) were deferred, and the documentation page is not yet reachable from the question editing form, which leaves part of NFR3 outstanding at the time of writing.

6.5 Future Work

The survey and the analysis point in the same direction for what to build next. The highest-value item is hints for hidden test cases (the deferred FR5): it was requested in the survey, it addresses the student-side pain the analysis rated most serious, and the literature identifies feedback quality as the field's persistent weakness [5]. The copy button on code blocks (FR9) remains a small, high benefit-to-cost addition. The ACE editor could be easily made to support linters and LSPs with some modification which is an a very good contribution on its own. Beyond the deferred requirements, three items emerged during the project: extending the layout switcher to other question types, which half of all survey respondents requested (however prompted); completing the screen reader announcement of test results, for which the groundwork is in place but the renderer must be restructured to mutate rather than replace its results container; and defining a moderated contribution model for the documentation page, answering the governance concern raised in the survey, so that experienced question authors can extend it without disputes over content. Finally, the changes themselves are candidates for contribution upstream to the CodeRunner project, which would expose them to a far larger user base and subject them to review by the tool's maintainers.

7 Conclusion

The conclusion is similar to when a plane lands. You don't rewrite the introduction. You say something like - I addressed the problem outlined in the introduction, and I built some software to do this. I tested the software like this ADD FEW WORDS.

Summarize main findings drawn from the project work. Mention the objectives or research questions. Do not repeat points raised in the discussion and reflection Section. If applicable, you can make recommendations. The conclusion should NOT contain any references.

References

- [1] CodeRunner. Coderunner usage documentation. https://trampgeek.github.io/moodle-qtype_coderunner.
- [2] Google. Google developer documentation style guide. <https://developers.google.com/style>.
- [3] O. C. Z. Gotel and A. C. W. Finkelstein. An analysis of the requirements traceability problem. In *Proceedings of the First International Conference on Requirements Engineering*, pages 94–101. IEEE, 1994.
- [4] P. Ihanola, T. Ahoniemi, V. Karavirta, and O. Seppälä. Review of recent systems for automatic assessment of programming assignments. In *Proceedings of the 10th Koli Calling International Conference on Computing Education Research*, Koli Calling '10, page 86–93, New York, NY, USA, 2010. Association for Computing Machinery.
- [5] H. Keuning, J. Jeuring, and B. Heeren. A systematic literature review of automated feedback generation for programming exercises. *ACM Transactions on Computing Education*, 19(1), 2018.
- [6] B. A. Kitchenham and S. L. Pfleeger. Principles of survey research part 2: designing a survey. *SIGSOFT Softw. Eng. Notes*, 27(1):18–20, Jan. 2002.
- [7] S. Krug. *Don't Make Me Think, Revisited: A Common Sense Approach to Web Usability*. New Riders Publishing, USA, 3rd edition, 2014.
- [8] R. Lobb and J. Harlow. Coderunner: a tool for assessing computer programming skills. *ACM Inroads*, 7(1):47–51, Feb. 2016.
- [9] MDN Web Docs. Progressive enhancement. https://developer.mozilla.org/en-US/docs/Glossary/Progressive_Enhancement.
- [10] D. Merkel. Docker: lightweight linux containers for consistent development and deployment. *Linux J.*, 2014(239), Mar. 2014.
- [11] M. Mihajlija and P. Walton. Cumulative layout shift (CLS). <https://web.dev/articles/cls>.
- [12] MITRE. CWE-22: Improper limitation of a pathname to a restricted directory ('path traversal'). <https://cwe.mitre.org/data/definitions/22.html>.
- [13] Moodle. Question behaviour plugins. <https://moodledev.io/docs/apis/pluginatypes/question/behaviour>.
- [14] D. Norman. *Design Of Everyday Things*. The MIT Press, 2013.
- [15] D. North. Introducing BDD. <https://dannorth.net/introducing-bdd/>, 2006.
- [16] T. Phung, V.-A. Pădurean, J. Cambronoero, S. Gulwani, T. Kohn, R. Majumdar, A. Singla, and G. Soares. Generative ai for programming education: Benchmarking chatgpt, gpt-4, and human tutors. In *Proceedings of the 2023 ACM Conference on International Computing Education Research - Volume 2*, ICER '23, page 41–42, New York, NY, USA, 2023. Association for Computing Machinery.
- [17] V. Pieterse. Automated assessment of programming assignments. In *Proceedings of the 3rd Computer Science Education Research Conference on Computer Science Education Research*, CSERC '13, page 45–56, Heerlen, NLD, 2013. Open Universiteit, Heerlen.

REFERENCES

- [18] W3C. Accessible rich internet applications (wai-aria) 1.2. W3c recommendation, World Wide Web Consortium, June 2023. <https://www.w3.org/TR/wai-aria-1.2/>.
- [19] W3C. Web content accessibility guidelines (wcag) 2.2. W3c recommendation, World Wide Web Consortium, Oct. 2023. <https://www.w3.org/TR/WCAG22/>.

A Appendix

Due to page count limitations, online only versions of the *participant sheet* and *consent form* can be found at the following links. Each respective section contains the same file; multiple hosts were used to provide backup sources. Note: The GitHub link may fail to load on the page, if so, download the raw file and open it with your favourite PDF viewer.

A.1 Ethical Approval Form

No files provided here.

ID	TITLE	STATUS	ACTION	RELATED IDS	LAST SUBMITTED	LAST REVIEWED
3295	Improving Coderunner	AP- PROVED	View Amend		2026-07-10 11:43:48	2026-07-10 15:01:20

Figure 10: Ethical Approval found on the Strathclyde Ethics page

A.2 Participant Information Sheet

Github: <https://github.com/michal-skraburski/mono-moodle-docker/blob/main/survey/information/Participant%20Information%20Sheet%20for%20Survey%20of%20Improving%20Coderunner.pdf>

A.3 Consent Form

Github: <https://github.com/michal-skraburski/mono-moodle-docker/blob/main/survey/information/consent%20form%20Coderunner.pdf>